

Experiment No.	5
Experiment Title	Decision Tree and Random Forest: A Comparative Classification Study ¹
Student Name	T.Ashwath Ram
Register Number	3122247001007
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	25.07.2026

1 Objective

- To implement a Decision Tree classifier.
- To extend the Decision Tree into a Random Forest ensemble model.
- To study the impact of hyperparameters on overfitting and generalization.
- To select optimal hyperparameters using 5-Fold Cross-Validation.
- To compare single-tree and ensemble-tree models.

2 Problem Statement

Given 30 numerical features computed from digitized images of a breast mass fine-needle aspirate (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension – each summarized by mean, standard error, and “worst”/largest value), the task is to predict whether the mass is **Malignant (M)** or **Benign (B)**. This is a binary classification problem. A single Decision Tree classifier is compared against a Random Forest ensemble, both tuned via 5-fold cross-validated Grid Search, to study how ensembling affects overfitting, variance, and generalization relative to a single tree.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository – https://archive.ics.uci.edu
Number of Samples	569
Number of Features	30 numerical attributes (10 base measurements × mean, standard error, and worst value)
Number of Classes	2 (Malignant = 1, Benign = 0) – class distribution: 357 Benign (62.7%), 212 Malignant (37.3%)
Missing Values	None (0 missing values confirmed across all 30 feature columns)

Train-Test Split	80% / 20%, stratified on diagnosis (455 training samples / 114 test samples), random_state = 42
------------------	---

4 Exploratory Data Analysis

- Dataset overview: 569 rows $\times$ 32 raw columns (id, diagnosis, 30 features); id dropped and `diagnosis` label-encoded (B $\rightarrow$ 0, M $\rightarrow$ 1) before modeling.
- Statistical summary: all 30 features are continuous, positive-valued measurements with widely differing scales (e.g. `area_mean` in the hundreds/thousands vs. `smoothness_mean` below 1), consistent with tree-based models being used here since they do not require feature scaling.
- Missing value analysis: confirmed zero missing values across all columns via `df.isnull().sum()`.
- Class distribution: see Figure 1.
- Correlation matrix: see Figure 2; 88 feature pairs were found with $|r| > 0.8$.

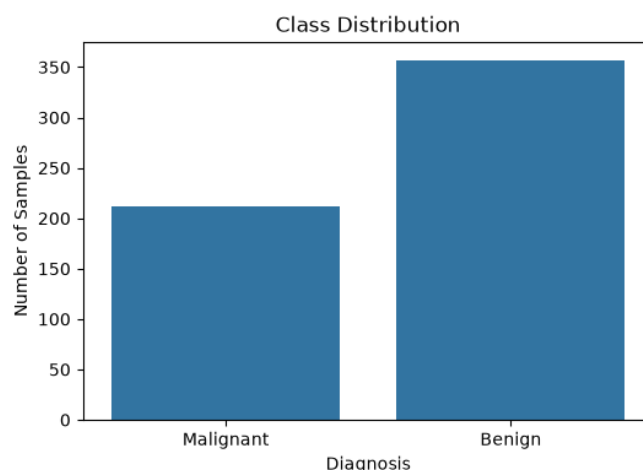


Figure 1: Class distribution – Benign vs. Malignant

Inference: The dataset shows a moderate class imbalance, with benign cases (62.7%) outnumbering malignant cases (37.3%). This imbalance is mild enough that accuracy remains a reasonably informative metric, but precision and recall are tracked separately, since in a diagnostic setting missing a malignant case (a false negative) is typically far more costly than a false positive.

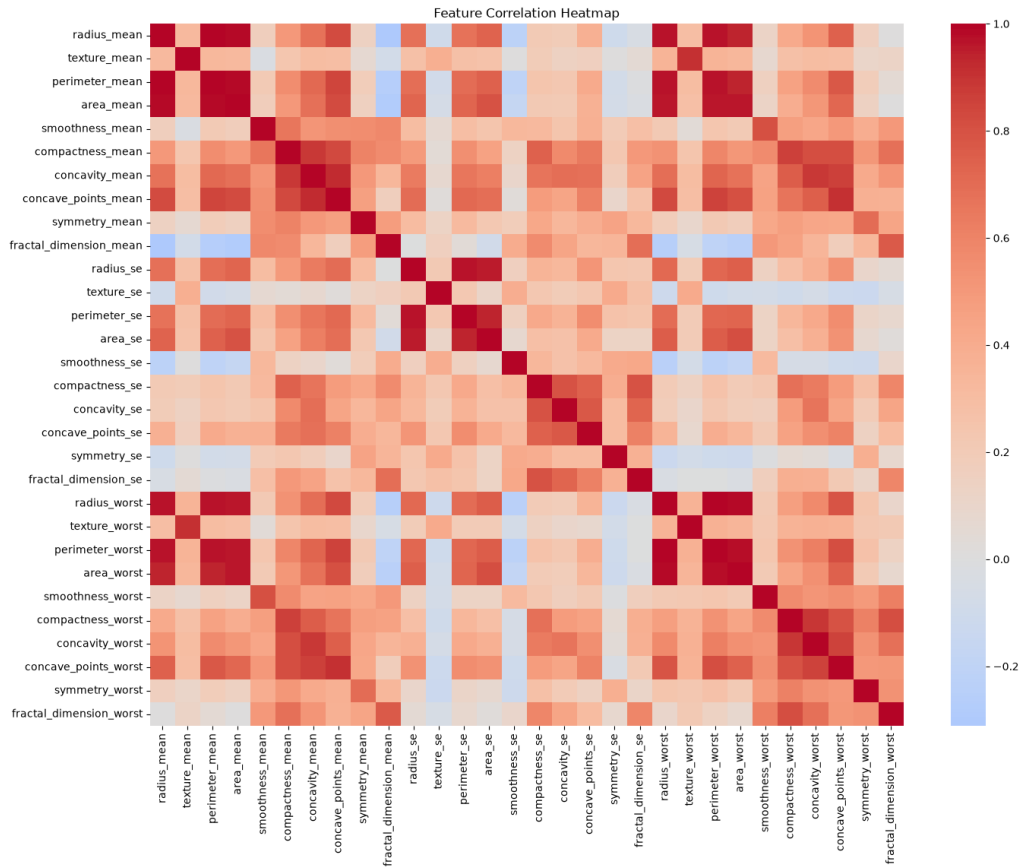


Figure 2: Feature correlation heatmap (30 numeric features)

Inference: Strong positive correlation clusters appear among the size-related features (radius, perimeter, area – each near-perfectly correlated with the other two within the same mean/se/worst group, e.g. $r = 0.998$ between `radius_mean` and `perimeter_mean`), which is expected since these three quantities are geometrically related. `concavity`, `concave_points`, and `compactness` also form a moderately correlated cluster. This high multicollinearity among size features is not a concern for tree-based models (unlike linear models), since trees split on one feature at a time and are insensitive to feature scale or correlation.

5 Data Preprocessing

- **Missing value handling:** Not required; zero missing values were confirmed across all columns.
- **Encoding:** The target `diagnosis` column was label-encoded (Benign $\rightarrow$ 0, Malignant $\rightarrow$ 1); the non-informative `id` column was dropped.
- **Feature scaling:** Not applied. Decision Trees and Random Forests split on raw feature thresholds and are invariant to monotonic feature scaling, so standardization was unnecessary (unlike the linear/kernel/distance-based models in earlier experiments).

- **Train-test split:** An 80/20 stratified split (`stratify=y`, `random_state=42`) was used, yielding 455 training and 114 test samples, preserving the class ratio in both.
- **Feature engineering:** None applied; all 30 original numerical attributes were used directly.

6 Machine Learning Algorithm

Decision Tree Classifier

Working principle: A Decision Tree recursively partitions the feature space by choosing, at each node, the feature and threshold that most reduces class impurity, forming a hierarchy of if-else decision rules down to leaf nodes that output a class prediction.

Mathematical formulation: Node impurity is measured by the Gini Index or Entropy:

$$\text{Gini} = 1 - \sum_{i=1}^C p_i^2, \quad \text{Entropy} = - \sum_{i=1}^C p_i \log_2 p_i \quad (1)$$

where p_i is the proportion of class i samples at a node. The tree greedily selects splits that maximize the impurity reduction (information gain) at each step. **Advantages:** Highly interpretable (rules can be read directly from the tree), requires no feature scaling, handles non-linear relationships naturally. **Limitations:** Prone to high variance and overfitting, especially as tree depth grows unconstrained – a single tree can memorize training noise, as seen later in this experiment’s depth-vs-accuracy analysis.

Random Forest Classifier

Working principle: Random Forest is an ensemble of many Decision Trees, each trained on a bootstrapped sample of the training data and considering only a random subset of features at each split; predictions are combined via majority voting.

Mathematical formulation: For B trees $\{T_b\}_{b=1}^B$, the ensemble prediction is

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \dots, T_B(x)) \quad (2)$$

with each T_b trained on a bootstrap sample $\mathcal{D}_b$ drawn with replacement from the training set, and each split restricted to a random subset of `max_features` features. **Advantages:** Reduces variance and improves generalization relative to a single tree by averaging over many decorrelated trees (bagging); less prone to overfitting. **Limitations:** Less interpretable than a single tree (a “black box” relative to one readable rule set), more computationally expensive to train and store, and can still overfit if individual trees are allowed to grow arbitrarily deep with insufficient regularization.

7 Experimental Setup

Python Version	3.13.15
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Windows
Hardware	Personal laptop (CPU-only)

8 Hyperparameter Selection

Parameter	Value
DT – criterion	gini, entropy
DT – max_depth	3, 5, 7, 10, None
DT – min_samples_split	2, 5, 10
DT – min_samples_leaf	1, 2, 4
RF – n_estimators	50, 100, 200
RF – max_depth	5, 10, 15, None
RF – max_features	sqrt, log2
RF – bootstrap	True, False

Both search spaces (90 combinations for the Decision Tree, 48 for Random Forest) were evaluated with `GridSearchCV` using 5-fold cross-validation, scored on accuracy (F1 also tracked).

Decision Tree Cross-Fold Results (Top Configurations)

Criterion	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
entropy	None	94.73	0.9297
entropy	10	94.73	0.9297
entropy	7	94.73	0.9297
entropy	7	94.51	0.9274
entropy	10	94.51	0.9274
entropy	None	94.51	0.9274
gini	7	94.29	0.9227

Note: the top three rows are tied at 94.73% CV accuracy – `criterion=entropy`, `min_samples_split=5`, `min_samples_leaf=2`, with `max_depth` $\in \{7, 10, \text{None}\}$ all scoring identically, since the tree naturally stops splitting well before reaching these depths once `min_samples_split/leaf` constraints bind. `GridSearchCV.best_params_` (used for all downstream evaluation) selected the `max_depth=7` configuration – `{criterion: entropy, max_depth: 7, min_samples_leaf: 2, min_samples_split: 5}` – as the single best estimator.

Random Forest Cross-Fold Results (Top Configurations)

n_estimators	Max Depth	Max Features	Avg CV Accuracy (%)	Avg CV F1 Score
200	10	log2	97.14	0.9618
100	None	log2	96.92	0.9592
100	15	log2	96.92	0.9592
100	10	log2	96.92	0.9592
50	10	sqrt	96.70	0.9561

Best configuration: `n_estimators=200`, `max_depth=10`, `max_features=log2`, `bootstrap=False`, CV accuracy 97.14%, CV F1 0.9618 – selected by `GridSearchCV.best_params_` with no ties among the top result.

9 Results

Decision Tree (Tuned) – Test Set

Metric	Value
Accuracy	0.9561
Precision	1.0000
Recall	0.8810
F1-score	0.9367
ROC-AUC	0.9396

Random Forest (Tuned) – Test Set

Metric	Value
Accuracy	0.9649
Precision	1.0000
Recall	0.9048
F1-score	0.9500
ROC-AUC	0.9940

5-Fold Cross-Validation Accuracy Comparison

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree	90.11	96.70	92.31	91.21	94.51	92.97
Random Forest	95.60	98.90	94.51	96.70	96.70	96.48

All values are accuracy percentages, computed via `StratifiedKFold` ($K=5$) on the training split using each model's best tuned hyperparameters.

For reference, the untuned baseline Decision Tree (default hyperparameters, depth 8, 24 leaves) achieved test Accuracy 0.9298, Precision 0.9048, Recall 0.9048, F1 0.9048 – substantially below the tuned tree, showing that hyperparameter tuning alone improved test accuracy by roughly 2.6 percentage points.

10 Result Visualization

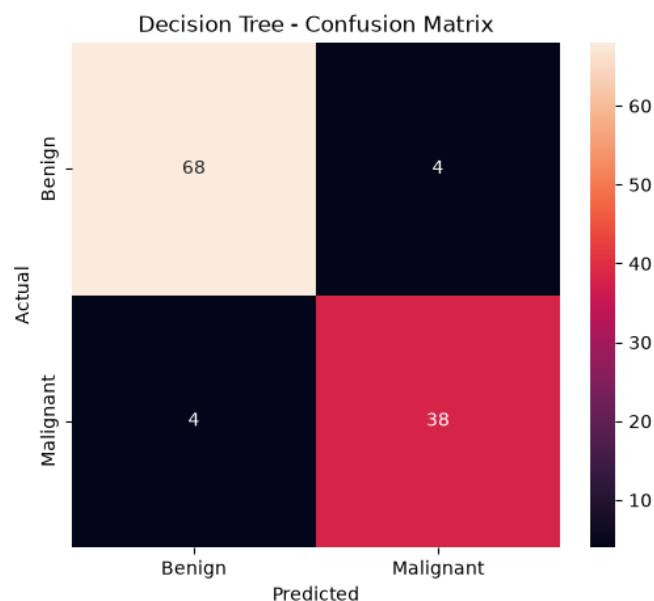


Figure 3: Confusion Matrix – Decision Tree (untuned baseline, default hyperparameters)

Inference: The untuned baseline tree makes a mix of both false positives and false negatives (roughly balanced), reflecting an unconstrained tree that has grown to depth 8 with 24 leaves – deep enough to fit considerable training noise. Its overall test accuracy (0.9298) is noticeably below the tuned tree’s, indicating that the default configuration is not optimal for this dataset.

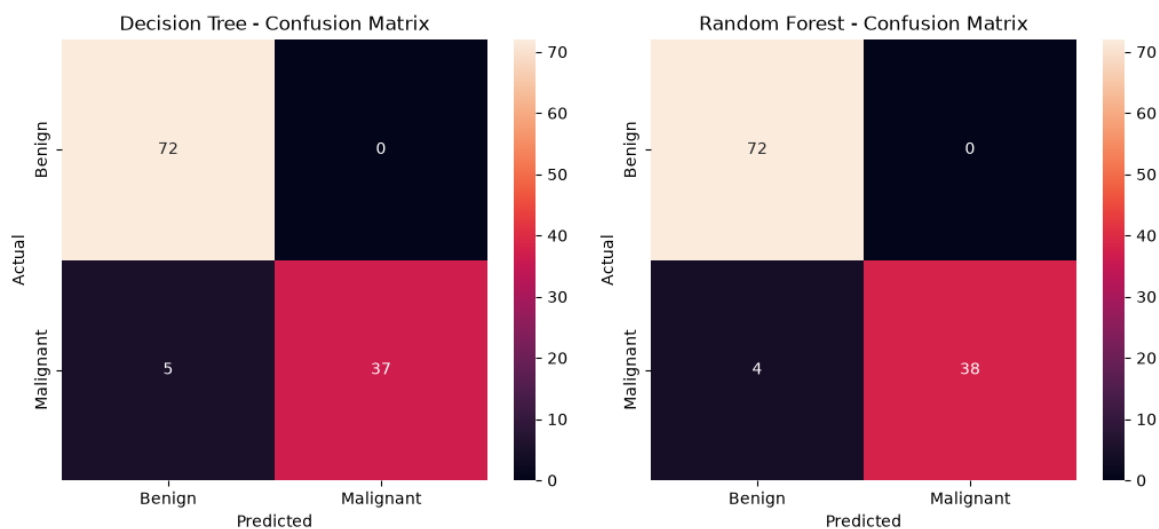


Figure 4: Confusion matrices – tuned Decision Tree (left) and tuned Random Forest (right)

Inference: Both tuned models achieve perfect precision on the malignant class (zero false positives – no benign case is ever misclassified as malignant), but the Decision Tree misses more true malignant cases (5 false negatives out of 42) than the Random Forest

(4 false negatives). Given that a false negative (missing a malignant tumor) is the more clinically costly error, the Random Forest's slightly better recall (0.9048 vs. 0.8810) is a meaningful practical advantage beyond what the accuracy difference alone suggests.

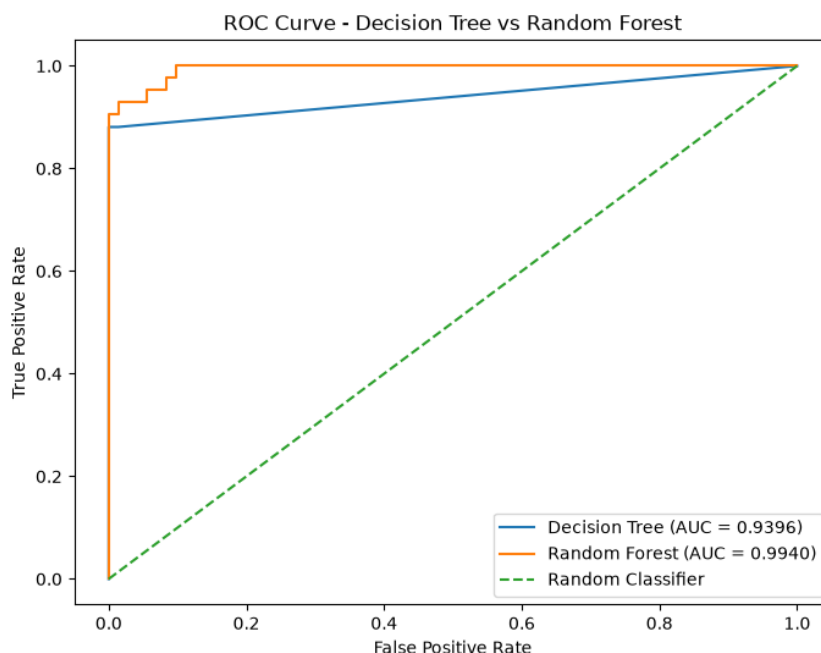


Figure 5: ROC Curve – Decision Tree vs. Random Forest

Inference: Random Forest's ROC curve dominates the Decision Tree's almost everywhere, with a near-perfect AUC of 0.9940 compared to the tree's 0.9396. This gap is far larger than the accuracy gap between the two models (0.9649 vs. 0.9561), showing that the Random Forest's predicted probabilities are much better calibrated/ranked than the single tree's – a benefit of averaging many trees' probability estimates that a single tree's more discrete, step-function-like probabilities cannot match.

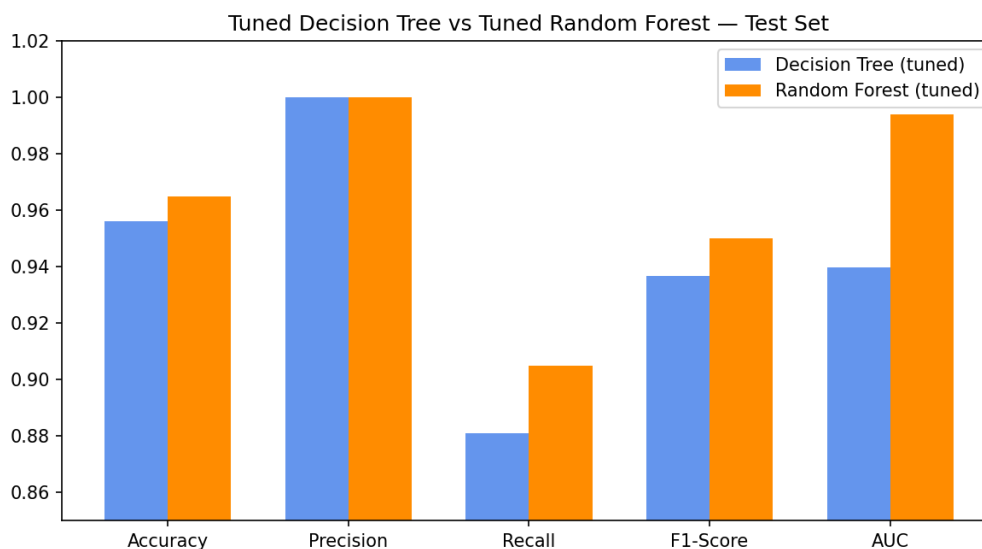


Figure 6: Tuned Decision Tree vs. tuned Random Forest – all test-set metrics

Inference: Random Forest matches or exceeds Decision Tree on every metric, with the largest gap in AUC and a smaller but consistent edge in accuracy, recall, and F1. Precision is tied at a perfect 1.0 for both models, meaning the accuracy/F1/recall differences are driven entirely by the malignant-class false negatives discussed above.

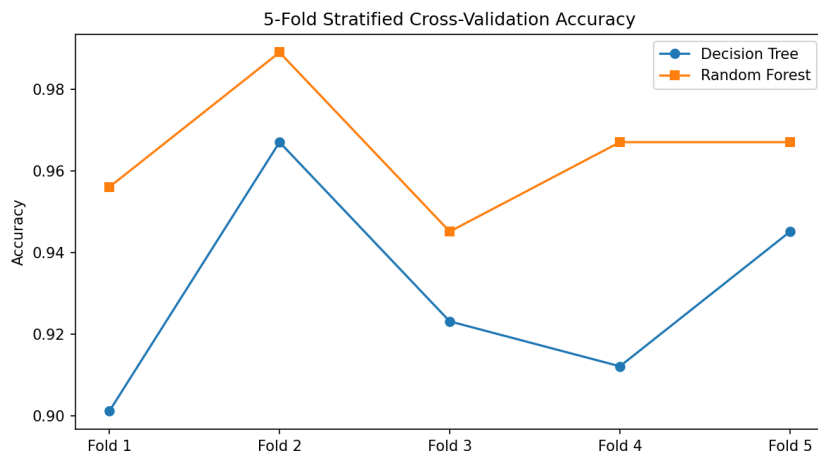


Figure 7: 5-fold cross-validation accuracy: Decision Tree vs. Random Forest

Inference: Random Forest outperforms the Decision Tree in every one of the 5 folds, with a notably smaller spread (95.60–98.90%) compared to the tree’s wider spread (90.11–96.70%). This lower fold-to-fold variance is the direct, expected effect of bagging – averaging over many bootstrapped trees smooths out the instability that a single tree exhibits when the training data changes slightly.

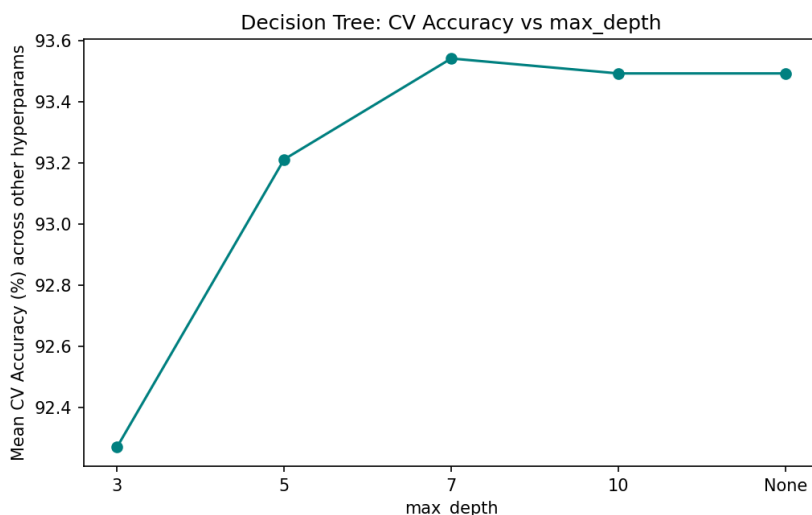


Figure 8: Decision Tree: mean CV accuracy vs. max_depth (averaged over all other hyperparameter combinations)

Inference: Accuracy rises sharply from depth 3 to depth 7 (92.27% to 93.54%), then plateaus and even dips slightly at depth 10 and beyond (including unbounded depth), showing that this dataset needs only a moderate tree depth (around 7) to capture the signal, and unconstrained depth offers no further benefit while adding overfitting risk –

consistent with the tied results at depths 7/10/None seen in the Table 1 results above once `min_samples_split/leaf` already limit further splitting.

11 Discussion

How does tree depth affect overfitting in Decision Trees?

The depth-vs-accuracy analysis shows CV accuracy improving up to depth ≈ 7 and then flattening, meaning depths beyond 7 add model capacity without improving (and slightly reducing) generalization – the classic overfitting signature. This is also visible in the baseline (untuned, default) tree, which grew to depth 8 with 24 leaves and underperformed the tuned, explicitly depth-limited tree on the test set.

Which hyperparameter had the greatest impact on performance?

For the Decision Tree, `max_depth` had the clearest effect (a 1.3 percentage-point swing from depth 3 to the plateau at depth 7+), with `criterion=entropy` consistently outperforming `gini` at matched depth in the top results. For Random Forest, `bootstrap=False` combined with `max_features=log2` dominated the top configurations, suggesting that using the full (non-bootstrapped) training set per tree combined with a smaller per-split feature subset generalized best on this particular dataset and split.

How does Random Forest improve generalization?

By training many trees on bootstrapped samples (or, in this experiment’s best configuration, varied feature subsets) and averaging their votes, Random Forest reduces the variance component of error without substantially increasing bias – visible directly in this experiment’s lower fold-to-fold spread (95.60–98.90% vs. 90.11–96.70% for the single tree) and higher AUC (0.9940 vs. 0.9396).

Did ensemble learning always improve performance? Why or why not?

Yes, in every measured comparison here – test accuracy, precision, recall, F1, AUC, and every individual cross-validation fold – Random Forest matched or exceeded the Decision Tree. This is not guaranteed in general (ensembling can fail to help, or even hurt, if the base trees are too correlated or the dataset is very small/simple), but for this 30-feature, 569-sample dataset with substantial feature redundancy (88 highly correlated pairs), the random feature subsampling in Random Forest had ample redundant signal to exploit for building diverse, decorrelated trees.

12 Conclusion

Both the tuned Decision Tree and tuned Random Forest achieve strong performance on the WDBC breast-cancer classification task, but Random Forest is the clear winner: higher test accuracy (0.9649 vs. 0.9561), higher recall on the clinically critical malignant class (0.9048 vs. 0.8810), a substantially higher ROC-AUC (0.9940 vs. 0.9396), and

lower variance across cross-validation folds. Grid Search identified `max_depth=7` (with `entropy`, `min_samples_split=5`, `min_samples_leaf=2`) as optimal for the single tree, and `n_estimators= 200`, `max_depth=10`, `max_features=log2`, `bootstrap=False` for the forest – both improving meaningfully over untuned defaults. The depth-vs-accuracy analysis confirms the expected bias-variance story: shallow trees underfit, depth around 7 is near-optimal for a single tree on this dataset, and further depth adds overfitting risk without benefit, a risk that Random Forest’s bagging/feature-subsampling largely neutralizes. For a real diagnostic application, Random Forest’s combination of higher recall and much better-calibrated probability ranking (AUC) makes it the preferable model, at the cost of the interpretability advantage a single Decision Tree retains.

13 References

1. Scikit-learn Developers, “Decision Trees,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/tree.html>
2. Scikit-learn Developers, “Ensemble methods – Random Forests,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html#random-forests>
3. Scikit-learn Developers, “Tuning the hyper-parameters of an estimator,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html
4. UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Data Set.” [Online]. Available: <https://archive.ics.uci.edu>